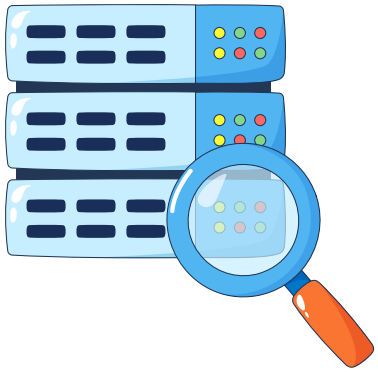
DynamoDB: Secondary Indexes and Querying

Explore how to efficiently retrieve data from Amazon DynamoDB using query and scan operations. Learn to optimize queries with filters and projections and understand how to design and maintain Local and Global Secondary Indexes for scalable access patterns.

Before building performant applications with DynamoDB, we need to understand how data is accessed and retrieved efficiently. This lesson focused on the fundamental operations and indexing strategies that shape how we interact with data at scale.



Query vs. scan

DynamoDB provides two primary operations to retrieve multiple items:

- **Query operation:** A query is used when we know the partition key ar to retrieve all (or a subset of) items that share that key. Queries can be

Ask

efficient, as DynamoDB uses the partition key to locate the appropriate data partitions and then applies optional filters to narrow the results further.

- **Scan operation:** A scan reads every item in the table, regardless of key. This makes scans significantly more resource-intensive, especially as the table size grows. Scans are often used during data migrations, audits, or administrative tasks, not during routine application queries.

> **Note:** When possible, we should always prefer a query over a scan, especially in latency-sensitive applications. Queries use partition key targeting, while scans consume more read capacity and take longer.

The illustration below depicts the impact of a scan on the provisioned throughput of a DynamoDB table. Notice that as the size of requests becomes nonuniform, throttling (shown by the rectangle bar) becomes more frequent.

Ask

Impact of scan on the provisioned throughput of a DynamoDB table

Here are some of the strategies to minimize the impact of a scan operation:

- **Reduce page size using the `Limit` parameter:** Setting a smaller page size allows scans to consume fewer read capacity units per request. This spreads out the load and introduces pauses between operations, which helps prevent throttling.
- **Use smaller, repeated scan requests:** Performing many small scan operations instead of a single large one reduces contention and allows other critical requests to proceed without interference.
- **Implement exponential backoff on throttled requests:** Configure your application to automatically retry throttled scan requests using exponential backoff to reduce pressure on the system.

Parallel scans

Parallel scans allow DynamoDB tables to be read more efficiently by dividing the data into multiple segments, each processed by a separate worker thread. This approach is ideal for large tables (typically 20 GB or more) when sequential scans are too slow and read capacity is underutilized.

While parallel scans improve performance, they can quickly consume provisioned throughput and potentially throttle other applications. Developers can tune performance using the `TotalSegments` parameter, starting with one segment per 2 GB of data and adjusting based on system limits and scan efficiency. Monitoring scan impact and balancing throughput usage are essential to ensure optimal operation without resource contention.

We should always design applications around key-based queries whenever possible for predictable performance.

Ask

Filters and projections

To further refine our queries or scans, we can use filter expressions and projection expressions. A filter expression limits the results returned to only those that match certain attribute conditions, such as status = ‘active’.

We can fetch all the posts from the last 24 hours using the filter in Python:

> **Note:** Filter expressions do not reduce the read capacity units (RCUs) used—DynamoDB reads all matching items first, then applies the filter before returning the results.

A projection expression, on the other hand, specifies which attributes should be returned in the response. This is especially useful for reducing payload size and network cost. For example, we may want to retrieve `userId` and `postTimestamp`, and `content` to display posts in the last 24 hours.

The Python code below uses filter and for this projection expression to get posts in the last 24 hours:

main.py Python

```python
import boto3
import time

# Create DynamoDB client
dynamodb = boto3.client(
    'dynamodb',
    region_name="us-east-1"
)

# Timestamp 24 hours ago
cutoff = int(time.time()) - 86400

# Scan with aliased attribute names
```

Ask

Python code for using filters

To summarize, while filters help narrow data logically, projections optimize payloads physically.

Indexing in DynamoDB

Indexes enable alternate access patterns when the primary key is insufficient for querying. DynamoDB supports two types of indexes: Local Secondary Indexes (LSIs) and Global Secondary Indexes (GSIs).

Let's understand indexing through the help of the `UserPosts` table we discussed in the earlier lesson. The illustration below shows the attributes of the table.

UserPosts table to store social media posts

Local Secondary Indexes (LSIs)

An LSI uses the same partition key as the base table but introduces a different sort key. This allows us to retrieve the same set of partitioned items using an alternative ordering or filtering attribute. For instance, assume we want to get all the social media posts with the image as the `PostType`.

A naive approach would be to scan the entire `UserPosts` table and filter the results based on the `PostType` attribute:

Ask

main.py Python

```python
response = table.scan(
    FilterExpression=Attr('postType').eq('image')
)
```

But this would be an operational overload, especially if our table consists of billions of records.

To address this problem, we can add an LSI using `UserId` and `PostType` to query by type of post per user. The code widget below contains Python code to create a `UserPosts` table with LSI.

main.py Python

```python
import boto3
import botocore.exceptions
import time

dynamodb_client = boto3.client(
    'dynamodb',
    region_name="us-east-1"
)

table_name = 'UserPosts'

table = dynamodb_client.create_table(
    TableName=table_name
```

Ask

Python boto3 code to create the Social media posts table with LSI

> **Note:** In Amazon DynamoDB, Local Secondary Indexes (LSIs) must be defined at table creation time and cannot be added later.

LSIs support strongly consistent reads, meaning we can be assured we're getting the latest committed value, which is critical for transactional applications. The illustration below shows the LSI:

UserPosts table with the Local Secondary Index

Global Secondary Indexes (GSIs)

GSIs allow querying using a completely different partition key and an optional sort key. For example, in a social media application, the base `UserPosts` table might be keyed by `UserId` (partition key) and `PostTimestamp` (sort key), which allows efficient queries for a user's posts in chronological order. However, what if we want to support a completely different access pattern, such as:

> "Show me the latest posts across all users that include the hashtag `#travel`."

Ask

This type of query cannot be efficiently performed using the base table's primary key structure because `UserId` is required in every query, and `hashtag` isn't a key

attribute at all.

To solve this, we can define a Global Secondary Index (GSI) that enables queries using an alternate partition key—in this case, `hashtag`. For more granular access, we may also include a sort key such as `PostTimestamp` to allow ordering by time.

The code snippet given below shows the code to update the table with the GSI.

main.py Python

```python
import boto3

dynamodb = boto3.client(
    'dynamodb',
    region_name="us-east-1"
)

response = dynamodb.update_table(
    TableName='UserPosts',
    AttributeDefinitions=[
        {'AttributeName': 'hashtag', 'AttributeType': 'S'},
        {'AttributeName': 'PostTimestamp', 'AttributeType': 'S'}
    ]
```

Python code to update table with GSI

The illustration below depicts the key changes to our table:

Ask

UserPosts table with a Global Secondary Index

Unlike LSIs, GSIs support eventually consistent reads only. They offer much greater flexibility and can be added after table creation. Because GSIs are asynchronously updated, there can be a slight lag in reflecting the latest data, which should be accounted for in time-sensitive applications.

Index maintenance and best practices

Indexes consume additional storage and write throughput. When we write to a table, DynamoDB must also update all corresponding indexes, which affects performance and cost. Therefore, we should:

- Avoid indexing attributes that change frequently.
- Use sparse indexing and create GSIs only on items that contain the indexed attributes.
- Keep projected attributes minimal to reduce index size.
- Monitor index metrics like `OnlineIndexPercentageProgress` and `ConsumedWriteCapacityUnits` using CloudWatch.
- Regularly assess whether indexes are actively used; unused GSIs can be deleted to save cost.

By carefully planning our index strategy, we ensure that our access patterns remain scalable and efficient.

Here is a quick flowchart to summarize the use of indexing, scan operation, and queries to fetch data:

Ask

Flow chart to decide with method to use to fetch the data

Conclusion

In this lesson, we examined how to retrieve data efficiently from Amazon DynamoDB using queries, scans, filters, projections, and indexes. Understanding these operations was critical to building performant and scalable applications. We compared query and scan operations, explored the role of filters and projections,

Ask